



UNIVERSIDADE FEDERAL DE ALAGOAS  
INSTITUTO DE COMPUTAÇÃO  
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO  
DISCIPLINA: PROGRAMAÇÃO 2

ALUNO: YURI RAPHAEL MOTA DE ARAÚJO BARBOSA

## RELATÓRIO DO MILESTONE 2

MACEIÓ, 26 DE JUNHO DE 2026

## 1. INTRODUÇÃO

Este relatório documenta as decisões arquiteturais e o design adotado no desenvolvimento do projeto Jackut, uma rede de relacionamentos inspirada no Orkut. O documento apresenta uma análise crítica da estrutura estabelecida no Milestone 1 e descreve o refatoramento realizado para suportar as novas funcionalidades exigidas no Milestone 2. As implementações mais recentes englobam as User Stories 5 a 9, adicionando o suporte a comunidades, novos tipos de relacionamentos (fãs, paqueras e inimigos) e a exclusão completa de usuários do sistema. O foco manteve-se no desenvolvimento rigoroso da lógica de negócio, assegurando que o sistema passe em 100% dos testes de aceitação do framework EasyAccept.

## 2. AVALIAÇÃO

A transição para o Milestone 2 exigiu uma reavaliação rigorosa do design do sistema. Foi necessário aplicar padrões de projeto para evitar o alto acoplamento e problemas com divisão de responsabilidades, garantindo que a aplicação evoluísse sem sacrificar a manutenibilidade do código.

### 2.1 VIRTUDES DO PROJETO

O maior destaque do design atual é a forte coesão e o baixíssimo acoplamento alcançados. A classe Facade atua estritamente seguindo o padrão de projeto *Facade*: ela não possui nenhuma regra de negócio interna, delegando as ações para controladores especialistas como o UsuarioController e o ComunidadeController. Destaca-se também a aplicação do conceito de Injeção de Dependência, onde a Facade inicializa as estruturas de dados (mapaUsuarios e mapaComunidades) e as injeta nos controladores.

Outro ponto altíssimo do projeto é o Encapsulamento Defensivo. Na entidade Usuario, os métodos de acesso a atributos sensíveis, como getAmigos() e getMinhasComunidades(), retornam coleções imutáveis (Collections.unmodifiableList e Collections.unmodifiableSet). Isso blindar os dados internos do usuário contra modificações indevidas. O uso da estrutura LinkedHashSet nas listas de amigos e comunidades garante a unicidade dos

elementos e mantém a ordem de inserção, o que facilita a previsibilidade exigida pelos testes automáticos.

O tratamento de erros também merece grande destaque, resolvendo problemas comuns de granularidade excessiva no lançamento de exceções. O sistema utiliza um pacote exclusivo (br.ufal.ic.p2.jackut.exceptions) que isola as representações de erros de domínio. A superclasse JackutException herda de RuntimeException (unchecked), o que evita a poluição das assinaturas de métodos com cláusulas throws e a proliferação desnecessária de blocos try/catch pela arquitetura. Além disso, o uso de escapes unicode (como \u00E1 para caracteres acentuados) em classes como LoginInvalidoException e SenhaInvalidaException previne erros de formatação (encoding) na execução do console.

## 2.2 FRAQUEZAS DO PROJETO

Durante a evolução do projeto, notou-se o risco iminente da formação de uma "God Class". Caso as validações de comunidades, amizade e inimizade permanecessem concentradas na entidade Usuario ou diretamente na Facade, as classes se tornariam difíceis de manter. O mesmo se aplicava à persistência de dados em arquivos. Misturar operações de I/O em disco com a lógica de negócio de redes sociais viola princípios básicos de Orientação a Objetos. Este cenário motivou a reformulação arquitetural detalhada a seguir.

**Refatoramento e Desenvolvimento Realizado** Para implementar as novas *User Stories* mantendo a qualidade do software, o seguinte refatoramento foi conduzido:

- **Comunidades e Mensageria (US 5, 6 e 7):** Foi criada a entidade Comunidade, que encapsula a gestão de seu próprio dono (donoLogin) e de seus membros. Para o envio de recados e mensagens, abstraímos o conteúdo através da classe auxiliar estática MensagemCaixa, aninhada dentro de Usuario. Essa estrutura armazena o texto e o login do remetente, organizando-se em uma fila (Queue<MensagemCaixa>) que simplifica a leitura sequencial e a posterior exclusão de rastros.

- **Novos Relacionamentos e Bloqueios (US 8):** As lógicas de fã, ídolo, paquera e inimizade foram centralizadas no `UsuarioController`. Implementamos o método privado `validarBloqueioInimizade()`, que intercepta e bloqueia interações (como o envio de recados ou convites) diretamente na raiz se o remetente for um inimigo declarado. O sistema também passou a detectar paqueras mútuas automaticamente, disparando um recado forjado do remetente fictício "Jackut" para notificar os envolvidos.
- **Remoção de Conta (US 9):** A funcionalidade de encerramento de conta exigiu a implementação de uma rotina de exclusão em cascata profunda (*Cascade Delete*). O método `removeUsuario` do `UsuarioController` agora aciona `removeRastrosDeRelacionamento` em todos os outros usuários, limpando amizades, fãs, ídolos, paqueras e apagando as `MensagemCaixa` enviadas pelo usuário deletado. Simultaneamente, o `ComunidadeController` aciona `removeRastrosDeUsuario` para deletar as comunidades nas quais o usuário era dono e removê-lo como membro das demais.
- **Isolamento da Persistência:** Toda a lógica de gravação e leitura de arquivos foi isolada no `PersistenciaController`. Utilizamos a API de Serialização de Objetos nativa do Java (`ObjectOutputStream` e `ObjectInputStream`) para gravar arquivos binários (.dat), desacoplando a camada de dados da lógica de domínio.
- **Validações Granulares:** Exceções precisas como `AutoAmizadeException`, `ContaJaExisteException` e `ConvitePendenteException` foram criadas para fornecer respostas exatas aos casos de falha previstos nos testes de aceitação.

### 3. CONCLUSÃO

A adoção de uma arquitetura baseada em controladores altamente coesos e entidades independentes mostrou-se essencial para a expansão do sistema Jackut. O refatoramento contínuo, a injeção de dependências e o isolamento completo da persistência não apenas garantiram o atendimento integral aos testes de aceitação exigidos,

como resultaram em um código limpo, de fácil manutenção e fundamentado nos pilares de um bom design orientado a objetos.